Lezione 4 - Spec-Driven Development: dai requisiti a una specifica agent-ready

Durata indicativa: 30-45 minuti

Posizione: Domain -> Requirements -> **Spec** -> Test -> Agent -> Verify

Domanda guida: come trasformiamo requisiti gia chiariti in un artefatto sufficientemente preciso da guidare un agente senza costringerlo a inventare il dominio?

Obiettivi

• Capire che cosa cambia passando dai requisiti a una specifica operativa.

• Distinguere Requirement, Spec, Design, Plan e Ticket.

• Capire che cosa significa davvero Spec-Driven Development in ambiente agentico.

• Confrontare to-spec, GitHub Spec Kit, OpenSpec, Kiro, Tessl e Superpowers senza confonderli tra loro.

• Imparare a riconoscere una specifica sufficientemente solida da consegnare a un coding agent.

1. Dove siamo nel workflow

Nella Lezione 2 abbiamo esplorato il dominio: linguaggio, eventi, regole, confini e unknown. Nella Lezione 3 abbiamo trasformato quella conoscenza in requisiti verificabili, senza farli degenerare in decisioni di implementazione.

Adesso serve un artefatto che sopravviva alla chat e possa essere letto da una sessione o da un agente che non ha partecipato alla discovery. Questo e il ruolo operativo della specifica.

```
Domain -> Requirements -> SPEC -> Design/Plan -> Tickets -> Implementation -> Verify
```

La spec non deve essere una nuova fase di invenzione. Deve consolidare decisioni gia autorizzate, rendere visibile cio che e fuori scope e predisporre il terreno per la verifica.

2. Requirement, Spec, Design, Plan e Ticket

Livello	Domanda principale	Esempio
Requirement	Che cosa deve essere vero?	La chiusura e rifiutata se esistono interventi aperti.
Spec	Che cosa costruiamo e come lo verifichiamo?	Definisce stati aperti, attori, esiti, edge case e non-goals.
Design	Come lo realizziamo?	Domain model, API, persistenza, error model.
Plan	In quale sequenza?	Prima proteggere il dominio, poi esporre la capability.
Ticket	Quale slice eseguibile?	Chiusura rifiutata per intervento pianificato, end-to-end.

Attenzione: questi confini non sono universali. Alcuni tool chiamano "spec" un file che include anche parti di design; altri separano requirements.md, design.md e tasks.md. La competenza importante non e imparare un nome, ma capire quale decisione stiamo prendendo e quale artefatto la deve contenere.

3. Perche la spec e diventata centrale con gli agenti

In una squadra umana molta conoscenza rimane tacita: conversazioni, storia del prodotto, convenzioni del repository. Un agente che parte in una nuova context window non possiede automaticamente questa memoria. Se gli diamo solo una richiesta vaga, il modello puo compensare con inferenze plausibili ma non autorizzate.

La spec diventa quindi una **interfaccia tra intent umano e implementazione agentica**. Ha tre funzioni: preservare decisioni, delimitare l autorita dell agente e offrire un punto stabile contro cui verificare il risultato.

Una buona spec e una forma di context engineering: non cerca di mettere tutto nel contesto, ma seleziona cio che deve sopravvivere alla conversazione.

4. Che cosa significa Spec-Driven Development

Spec-Driven Development (SDD) non e oggi una metodologia universalmente standardizzata. E una famiglia di approcci nei quali la specifica diventa un artefatto centrale del ciclo di sviluppo e viene usata attivamente da persone e agenti.

GitHub Spec Kit descrive un core SDD basato su Specify -> Plan -> Tasks -> Implement -> Converge e su artefatti Markdown che alimentano le fasi successive. La filosofia dichiarata e intent-driven: definire il "what" prima del "how" e raffinarlo attraverso piu passaggi.

OpenSpec separa proposal, specs, design e tasks; nel workflow predefinito il proposal viene prima, specs e design possono essere prodotti in parallelo, poi arrivano tasks e apply. Il design puo essere omesso quando non porta valore.

Kiro usa Feature Specs con requirements, design e tasks. Supporta un percorso Requirements-First e uno Design-First; nella fase requisiti usa EARS e offre anche analisi delle ambiguita e incoerenze.

Tessl distribuisce una metodologia SDD come tile composto da skills, rules, docs, scripts ed evals: requirement gathering, spec writing, approval, implementazione, review e verifica.

5. Quanto forte deve essere il ruolo della spec?

Per ragionare sul tema usiamo una tassonomia didattica, non uno standard:

Spec-first - La spec guida una change ma puo perdere centralita dopo il merge.

Spec-anchored - La spec rimane versionata e viene riconciliata quando la feature cambia.

Spec-as-source - La spec aspira a diventare la fonte primaria da cui codice e altri artefatti vengono derivati o rigenerati.

Non esiste una risposta unica. Piu autorita assegni alla spec, piu devi investire in versionamento, review, drift detection e verifiche automatiche.

6. Quando una spec e agent-ready

■ Scope chiaro e non-goals espliciti.

■ Vocabolario coerente con il domain model.

■ Regole e invarianti esplicite, non nascoste in prose ambigue.

■ Acceptance examples rappresentativi, inclusi casi negativi.

■ NFR misurabili oppure dichiarati UNKNOWN se la soglia non e autorizzata.

■ Dipendenze, vincoli e precondizioni visibili.

■ Uno o pochi punti di verifica/test seam ragionevoli.

■ Nessuna decisione inventata per riempire un template.

■ Coerenza interna sufficiente da non produrre due implementazioni incompatibili.

■ Dimensione compatibile con il context budget dell agente e con la decomposizione successiva.

7. Authority preservation e traceability

La formalita non crea autorita. Se durante il discovery avevamo ASSUMED o UNKNOWN, inserirlo in una sezione "Requirements" non lo trasforma magicamente in una decisione.

```
Evidence -> Requirement -> Spec -> Acceptance Example -> Test -> Implementation
```

Ogni passaggio dovrebbe poter essere ricondotto al precedente. Questa catena aiuta a distinguere regole reali da dettagli introdotti accidentalmente dall agente.

8. Esempio completo: chiusura di un ticket

Domain rule: un ticket chiuso non puo avere interventi operativamente aperti.

Requirement: quando un soggetto autorizzato richiede la chiusura, il sistema deve verificare che non esistano interventi aperti.

Spec: definisce quali stati contano come "aperti", chi puo tentare la chiusura, cosa osserva il chiamante quando la richiesta e rifiutata, come trattare il caso di ticket senza interventi, quali eventi di audit sono richiesti, quali comportamenti sono esplicitamente fuori scope e attraverso quale seam sara verificato il comportamento.

Design: decide se l invariante vive in un aggregate, application service o altro boundary; definisce endpoint, schema, persistenza, transazioni e integrazioni.

Ticket: produce una slice verticale verificabile, per esempio "rifiutare la chiusura quando esiste un intervento PIANIFICATO" attraversando domain, application, API e test necessari.

9. Matt Pocock: to-spec non e discovery

Nel workflow attuale delle engineering skills di Matt Pocock, **to-spec** e un passo del ramo multi-sessione. La documentazione esplicita che dovrebbe scrivere la spec a partire da decisioni gia prese e che non deve condurre una nuova intervista o inventare contenuti per riempire il documento.

```
grill-with-docs -> to-spec -> to-tickets -> implement -> code-review
```

to-tickets trasforma il lavoro in tracer bullets: slice strette ma complete che attraversano gli strati necessari e che possono essere verificate singolarmente. I blocking edges rendono visibile quali ticket sono pronti e quali dipendono da altri.

La stessa documentazione segnala che, se il cambiamento e piccolo e sta in una sola context window, e ragionevole saltare il ramo spec/ticket e passare direttamente a implement. Questa e una lezione generale: il processo deve essere proporzionato al rischio e alla durata del lavoro.

10. Confronto operativo dei principali approcci

Approccio	Categoria	Artefatti/flow	Quando e utile
Matt Pocock	Skill workflow	grill -> spec -> tracer tickets -> implement -> review	Quando vuoi skill piccole, composabili e context hygiene esplicita.
GitHub Spec Kit	Toolkit/process harness	Specify -> Plan -> Tasks -> Implement -> Converge	Quando vuoi un processo strutturato, estensibile e agent-agnostic.
OpenSpec	Framework/CLI leggero	Proposal -> Specs/Design -> Tasks -> Apply -> Archive	Quando vuoi change management in-repo e una separazione chia
Kiro	IDE + spec agents	Requirements -> Design -> Tasks	Quando vuoi un ambiente integrato con review gates, EARS e ana
Tessl	Registry + skill/rule package	Gather -> Spec -> Approve -> Implement -> Review -> Verify	Quando vuoi distribuire metodologia e context come pacchetti vers
Superpowers	Metodologia a skill componibili	Brainstorm/design -> plan -> TDD/subagents -> verify	Quando vuoi una disciplina agentica completa con forte enfasi su T

11. Tool per tool: problema, principio, quando usarlo

GitHub Spec Kit

Problema: prompt ad hoc e processi inconsistenti. Principio: intent prima dell implementazione, artefatti strutturati e analizzabili. Usalo se vuoi un workflow estensibile con integrazione a molti agenti. Evitalo se la change e minuscola o se il team non vuole mantenere artefatti di processo. Alternativa: OpenSpec o skill workflow piu leggeri.

OpenSpec

Problema: chat history effimera e difficolta a gestire cambiamenti brownfield. Principio: ogni change possiede un proposal, delta di spec, eventuale design e tasks. Usalo quando vuoi che il repository conservi il perche e il cosa di ogni change. Evitalo se vuoi un processo completamente conversazionale e senza file di supporto.

Kiro

Problema: passare da descrizione naturale a requisiti, design e task verificabili in un IDE integrato. Principio: approval gates e sincronizzazione tra artefatti. Usalo quando vuoi guidare utenti meno esperti o quando EARS e requirement analysis aiutano. Evitalo se vuoi massima portabilita tra harness diversi.

Tessl

Problema: distribuire e valutare il context operativo che guida gli agenti. Principio: skills/rules versionabili + evals per misurare l impatto. Usalo quando il team vuole trattare il processo agentico come un pacchetto condiviso. Alternativa: repository di skills personalizzate o Superpowers.

Superpowers

Problema: agenti che saltano discovery, TDD o verifica. Principio: systematic workflows over ad-hoc moves, evidence before claims. Usalo se vuoi una metodologia composta da skill operative che si attivano nel ciclo di sviluppo. Non e un sostituto del domain modeling: puo orchestrare bene solo cio che e stato realmente chiarito.

12. Anti-pattern della specificazione

Mega-spec: un documento cosi grande che nessun agente riesce a tenerne il modello mentale intero.

Spec theater: documenti che esistono per forma ma non guidano task, test o review.

Design leakage: la spec incorpora framework, classi o database decision senza che siano veri vincoli.

Pseudo-precisione: numeri e soglie inventati per sembrare rigorosi.

Spec drift: il codice cambia e la spec resta ferma, oppure viceversa.

Template worship: compilare sezioni senza valore solo perche il template le prevede.

Agent authority escalation: un agente trasforma una domanda aperta in una policy senza fonte.

13. Quando NON serve una spec completa

Un processo SDD completo ha un costo cognitivo e manutentivo. Va usato quando riduce un rischio reale: alta densita di decisioni, molti attori, NFR importanti, piu sessioni/agenti, forte auditabilita o alto costo di errore.

Per typo, refactoring meccanici, piccoli bug con test che gia definiscono il comportamento o cambi locali altamente reversibili, puo bastare un intent chiaro, un test e una review. La regola non e "sempre spec", ma **preserva abbastanza contesto da rendere improbabile un interpretazione non autorizzata**.

14. Esercizio pratico - 25 minuti

Riprendi la capability "Chiusura Ticket" o quella usata nelle lezioni precedenti.

• **Minuti 0-5:** scrivi Scope, Out of scope e Glossario minimo.

• **Minuti 5-10:** inserisci 3-5 regole/invarianti e 3 acceptance examples, incluso almeno un caso negativo.

• **Minuti 10-15:** aggiungi un NFR giustificato oppure marcato UNKNOWN e individua una test seam.

• **Minuti 15-20:** separa Spec da Design. Sposta fuori dalla spec classi, framework, tabelle, endpoint e pattern che non siano vincoli reali.

• **Minuti 20-25:** decomponi la spec in 2-4 tracer bullets. Ogni ticket deve produrre una slice dimostrabile e avere criteri di verifica propri.

Controllo finale: ogni MUST ha una fonte? ogni UNKNOWN e ancora visibile? ogni ticket consegna un comportamento osservabile? la spec contiene solo dettagli tecnici autorizzati?

15. Takeaway

Requirements dicono che cosa deve essere vero. La spec rende quell intent coerente, bounded, durevole e verificabile. Il design decide come soddisfarlo. I ticket trasformano il design in lavoro agent-ready. La verification controlla che il risultato sia allineato non solo al codice atteso, ma all intent originario.

Una buona spec non contiene il massimo numero di dettagli: contiene abbastanza informazione autorizzata da rendere difficile un interpretazione sbagliata.

Prossima lezione: TDD, BDD, ATDD e Specification by Example - trasformare la spec in prove eseguibili senza ridurre il prodotto alla sola suite di test.

Fonti e letture

Matt Pocock - to-spec
https://github.com/mattpocock/skills/blob/main/docs/engineering/to-spec.md

Matt Pocock - to-tickets
https://github.com/mattpocock/skills/blob/main/docs/engineering/to-tickets.md

GitHub Spec Kit - docs
https://github.github.com/spec-kit/

GitHub Spec Kit - What is SDD?
https://github.com/github/spec-kit/blob/main/docs/concepts/sdd.md

OpenSpec - spec-driven schema
https://openspec.dev/docs/schemas/spec-driven

Kiro - Specs best practices
https://kiro.dev/docs/specs/best-practices/

Kiro - first project/specs
https://kiro.dev/docs/getting-started/first-project/

Tessl - Spec-Driven Development tile
https://tessl.io/registry/tessl-labs/spec-driven-development/2.0.1/files/docs/index.md

Superpowers - repository
https://github.com/obra/superpowers